Spring Boot Annotations

1. ***@SpringBootApplicaiton ->*** It is main annotation to run spring boot application
    - ***@EnableAutoConfiguraiton*** -> This annotation helps to spring boot application to configure classes/interfaces which are available in class path in the form of jar.
      Ex: h2 jar -> h2 database related classes automatically configure in spring boot application.
    - ***@ComponentScan*** -> If helps IOC container to scan beans which are present in Spring Boot application. We can specify the package/base package/base classes.
    - ***@Configuration*** -> It helps to load java classes. We specify bean definitions inside it.

Stereotype annotations -> These annotations are used to create spring bean automatically in application context. Spring IOC will manage bean life cycle (bean creation to bean destroy).

2. ***@Component ->*** It is base annotation. Others are derived from @Component

3. ***@Service - >*** This annotation is used to write service layer (business logic)

4. ***@RestController/@Controller ->*** This annotation is used to write controller layer where API endpoints are defined.

5. ***@Repository ->*** This annotation is used to write persistent layer where actual database calls are happening.

Spring Core Annotations -> These are spring core annotations.

6. ***@Configuration*** – This annotation is used to by Spring IOC to read the beans which are there in BeanConfig class. – If you want define and provide a bean to Spring IOC then we write those beans under the class which is annotated with @Configuration.

7. ***@Bean ->*** If you want to provide a bean (Java Object) to spring IOC then we use @Bean on top of method.

8. ***@Autowired*** -> It helps to inject Object in dependent classes – IOC create implementation object of the interface and inject it.

9. ***@Qualifier("xxxx")*** -> It provides the one implementation class object among multiple implementation classes.

10. ***@Primary*** -> It is alternative of *@Qualifier* annotation. - > We can add @Primary on one implementation class among multiple implementation classes.

11. ***@Lazy*** → By default for all beans Spring container will create objects when application started. But if you want tell Spring container, create bean object when we use it. Then we need @Lazy annotation on top of the class.

12. ***@Value*** **->** It helps us to assign(set) values to properties(fields) from application.properties file
    Ex : @Value("${mail.from}")
    private String from;

13. ***@PropertySource*** -> By default spring boot will load properties from application.properties/yaml file. But if you want load from values from custom properties file then we have to use @PropertySource annotation.

    @PropertySoruce("classpath:custom.properties")
    Ex : public class Student

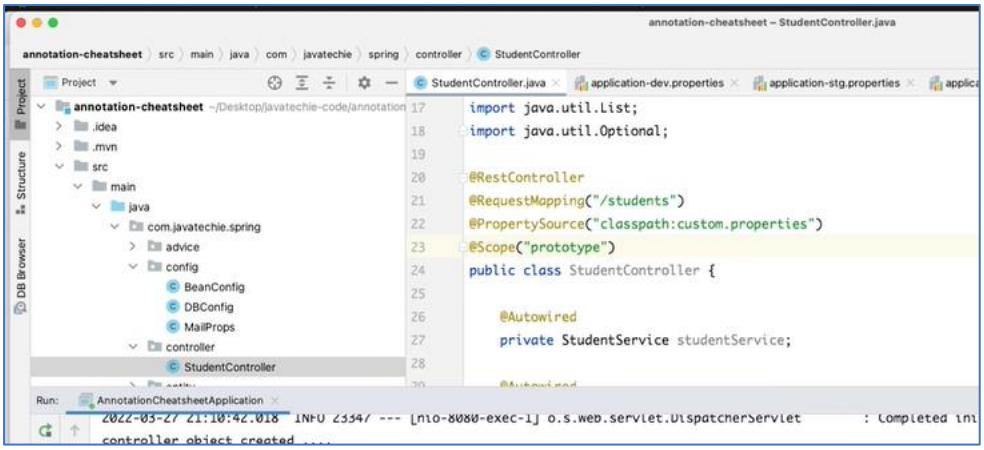
14. ***@ConfigurationProperties ->*** If you want inject values from properties file with some prefix then we will use @ConfigurationProperties.
    Ex : @ConfigurationProperties(prefix = "student")
    Public class Student → All properties which prefixed student are loaded into student class. Properties names and key names should match here.

```java
import lombok.Data;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

@ConfigurationProperties(prefix = "mail")
@Data
@Component
public class MailProps {

    private String from;
    private String host;
    private String port;
}
```

15. ***@Profile*** -> It helps to load properties for multiple environments.
    Ex : Set **spring.profiles.active** = xxx in application.properties
    Set @Profile annotation on top of the method wherever you are printing values

```java
@Bean
@Profile("dev")
public String dataSourceProps() {
    System.out.println(driverClass + " : " + url + " : " + username + " : " + password);
    return "dataSource connection for dev";
}
```

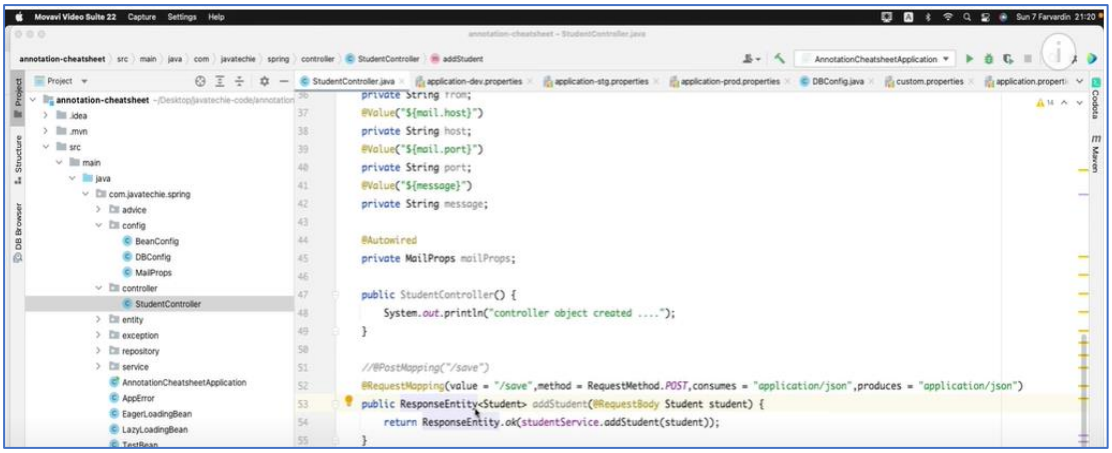
16. ***@Scope -*** > Based on the scope value objects will be created for context.



- **Singleton** -> Only object will be created for context. It is default scope.
- **Prototype** - > For each request object will be created
- **Request** -> This scopes a bean definition to an HTTP request. Only valid in the context of a web-aware Spring ApplicationContext.
- **Session** -> This scopes a bean definition to an HTTP session. Only valid in the context of a web-aware Spring ApplicationContext.
- **Global-session** -> This scopes a bean definition to a global HTTP session. Only valid in the context of a web-aware Spring ApplicationContext.

REST API Related Annotations -> These are spring core annotations.

17. ***@RestController ->*** It is combination of @Controller and @ResponseBody. It indicates that the class having REST endpoints.
18. ***@RequestMapping*** -> it is used to map http requests. It can be used for classes and methods.



19. ***@GetMapping –>*** To get data (select)

20. ***@PostMaping - >*** To create data. It is not idempotent which means multiple objects can create at backend if you hit request multiple times.

21. ***@PutMapping ->*** To update data -> It is idempotent.

22. ***@DeteteMapping ->*** To delete data -> It is idempotent.

23. ***@RequestBody*** -> It will be used in POST method to create data. It helps us to de-serialize http request into corresponding object/model.

24. ***@PathVariable*** -> If you want pass the data as part of the request url.

25. ***@RequestParam*** -> If you want extract data from URL. Then you should go for @RequestParam

    https://www.baeldung.com/spring-requestparam-vs-pathvariable

    While ***@RequestParam*s extract values from the query string, *@PathVariable*s extract values from the URI path**:

    ```
    @GetMapping("/foos/{id}")
    @ResponseBody
    public String getFooById(@PathVariable String id) {
        return "ID: " + id;
    }
    ```

    Then we can map based on the path:

    ```
    http://localhost:8080/spring-mvc-basics/foos/abc
    ----
    ID: abc
    ```

```
@GetMapping("/foos")
@ResponseBody
public String getFooByIdUsingQueryParam(@RequestParam String id) {
    return "ID: " + id;
}
```

which would give us the same response, just a different URI:

```
http://localhost:8080/spring-mvc-basics/foos?id=abc
----
ID: abc
```

```
@GetMapping({"/myfoos/optional", "/myfoos/optional/{id}"})
@ResponseBody
public String getFooByOptionalId(@PathVariable(required = false) String id){
    return "ID: " + id;
}
```

Then we can do either:

```
http://localhost:8080/spring-mvc-basics/myfoos/optional/abc
----
ID: abc
```

or:

```
http://localhost:8080/spring-mvc-basics/myfoos/optional
----
ID: null
```

26. ***@RestControllerAdvice & @ExceptionHandler*** -> These annotations are used to handle the exceptions. Whenever there is an exception in application then controller will delegate to @RestControllerAdvice mentioned class. And then we can Handle different time of exceptions with the help of @ExceptionHandler.

```
@RestControllerAdvice
public class StudentExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<AppError> handleException(StudentNotFoundException exception) {
        AppError error=new AppError(UUID.randomUUID().toString(),
                exception.getMessage(),
                HttpStatus.INTERNAL_SERVER_ERROR);
        return new ResponseEntity<>(error,HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Spring Data JPA Related Annotations -> These are Spring data JPA annotations.

27. ***@Entity ->*** for class we use this annotation to create table in database

28. ***@Table ->*** If you want provide table name in the database then we use this annotation

29. ***@Column*** -> If you want provide column names in the table then we use this annotation. Otherwise the column names will be created as property name.

30. ***@Id*** -> To create Primary key then we use this annotation.

31. ***@GeneratedValue*** -> to generate specific value to the column.

32. ***@Transcational - >*** It support to write Transaction Management code

Entity class Relationships

@OneToOne
@OneToMany
@ManyToOne
@ManyToMany